目 录

造像类文物三维建模自动化系统 4

代码说明书（P0 底座 · 第一先导底座） 4

第 1 章 概述 5

1.1 文档目的 5

1.2 P0 底座在系统中的地位 5

1.3 设计依据 5

第 2 章 模块划分 6

2.1 模块清单 6

2.2 模块划分原则 6

第 3 章 依赖关系与数据流 7

3.1 模块依赖关系图 7

3.2 包间依赖矩阵 7

3.3 一次运行的数据流 7

第 4 章 core/exceptions.py — 异常层级 9

4.1 功能 9

4.2 类层级 9

4.3 关键类/函数职责 9

4.4 错误码对照表 9

第 5 章 core/config.py — 配置加载与校验 10

5.1 功能 10

5.2 内部结构 10

5.3 校验行为 11

第 6 章 core/logging.py — 分级日志 12

6.1 功能 12

6.2 关键函数 12

6.3 设计要点 12
第 7 章 core/metadata.py — SHA256 与 run_manifest 溯源 13
7.1 功能 13
7.2 关键类/函数 13
7.3 RunManifest 关键方法 13
7.4 run_manifest.json 结构 13
第 8 章 cli.py — 命令入口与编排内核 15
8.1 功能 15
8.2 子命令 15
8.3 关键符号 15
8.4 编排内核逻辑（cmd_run） 15
8.5 异常退出码 15
第 9 章 关键接口契约汇总 17
第 10 章 目录与文件清单 18
10.1 项目目录（完整软著版骨架） 18
10.2 代码规模 18
第 11 章 测试与运行 19
11.1 运行单元测试 19
11.2 运行 CLI 19
11.3 测试覆盖 19
附录 A 配置字段速查 20
附录 B 合规与未核实事项 21

造像类文物三维建模自动化系统
代码说明书

（P0 底座 · 第一先导底座）

软件名称：石窟寺造像三维数据批量数字化处理系统（h3d-pipeline / 路线 E）

文档类型：代码说明书（Code Manual）

文档版本：V1.0 ｜ 编制日期：2026 年 8 月

对应代码：src/h3d/core/ + src/h3d/cli.py（P0 底座）

依据《软件开发项目设计书》《管线骨架构建分析》《开发指南》编制

造像类文物三维建模自动化系统

代码说明书（P0 底座 • 第一先导底座）

项目	内容
软件名称	石窟寺造像三维数据批量数字化处理系统（h3d-pipeline / 路线 E）
文档类型	代码说明书（Code Manual）
文档版本	V1.0
编制日期	2026 年 8 月
适用对象	系统架构师、算法/软件工程师、测试工程师、软著审查人员
对应代码	src/h3d/core/ + src/h3d/cli.py（P0 底座）

第 1 章 概述

1.1 文档目的

本说明书对「造像类文物三维建模自动化系统」管线的 **P0 底座（第一先导底座）** 的代码实现进行逐模块说明，内容包括：模块划分、关键类/函数的职责、内部数据结构、数据流与依赖关系，以及运行方式与测试方法，供开发、测试、维护与软件著作权登记使用。

1.2 P0 底座在系统中的地位

系统采用五层架构（《软件开发项目设计书》第 10 章），P0 底座对应其中的 L0–L1 依赖与 L4 应用入口，是全部业务子包（io / preprocess / register / change / colour / runner / report）的上游支撑：

```
L4 应用层   cli.py              命令入口（init / run / validate / report）   ← 本说明书
L3 编排层   runner/               批量调度、断点续跑、失败隔离
L2 业务层   preprocess/ register/ change/ colour/ report/
L1 适配层   io/                 点云与网格读写、格式适配
L0 依赖层   Open3D / NumPy / py4dgeo / colour-science（外部 MIT/BSD 库）
            core/               配置、日志、元数据、异常（P0 底座）        ← 本说明书
```

P0 底座由 **core 子包 + cli.py** 构成，职责为：参数校验与配置加载、分级日志、SHA256 校验与 run_manifest 溯源、自定义异常层级、命令行入口与编排内核。core 不依赖任何业务包，可被单独测试（见《管线骨架构建分析》第 5.2 节依赖矩阵）。

1.3 设计依据

- 《软件开发项目设计书》第 31 章（M0 命令与编排）、第 13 章（接口设计）、第 12 章（数据架构）、第 41 章（数据字典）
- 《管线骨架构建分析》第 4.1 节（P0 底座职责）、第 5 章（依赖关系分析）、第 6 章（占位文件与生成逻辑）
- 《开发指南》第 4.4.1 节（M0 入口与 CLI）、第 4.3 节（配置与溯源）

第 2 章 模块划分

2.1 模块清单

P0 底座包含 6 个源码文件，职责划分如下：

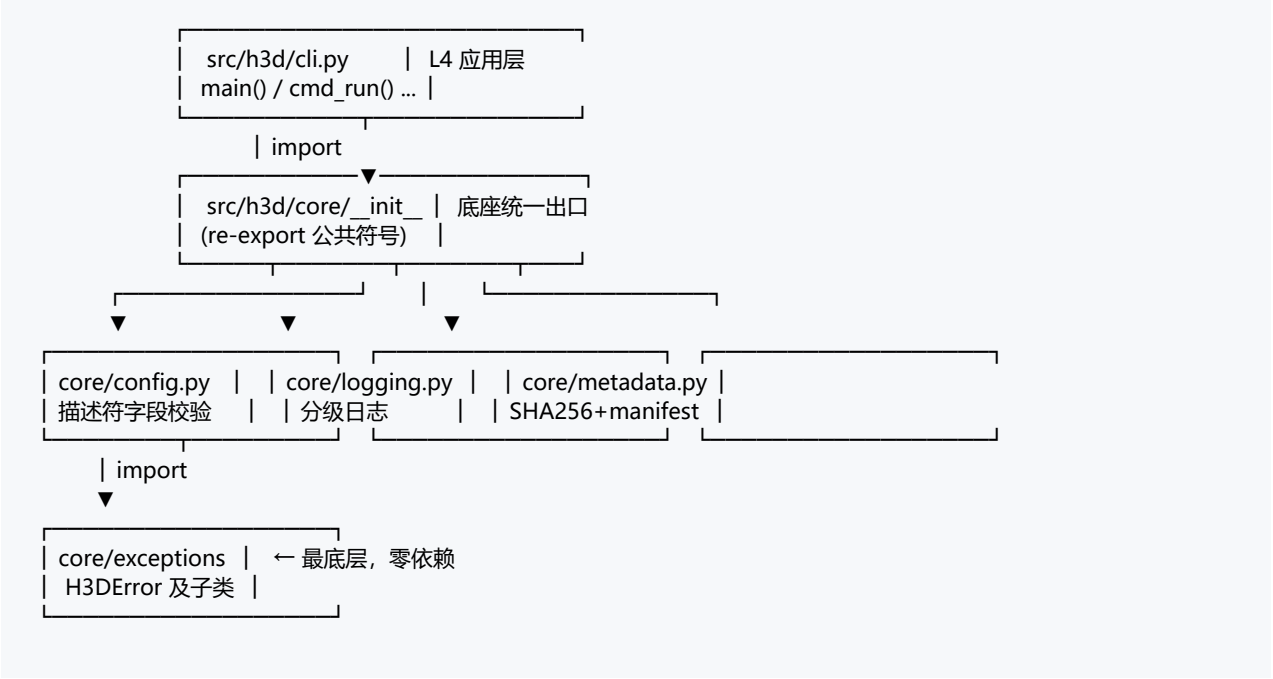
文件	模块标识	职责	依赖（导入方 → 被导入方）
src/h3d/__init__.py	包入口	声明包版本 __version__	无
src/h3d/core/exceptions.py	异常层	管线自定义异常类层级（错误码 E001/E100/E200/E500/E900）	无（最底层）
src/h3d/core/config.py	配置	描述符元编程字段校验 + YAML 加载 + config_sha256	exceptions, yaml
src/h3d/core/logging.py	日志	分级日志、控制台/文件双通道	logging(标准库)
src/h3d/core/metadata.py	溯源	SHA256 校验 + run_manifest 溯源元数据	dataclass, hashlib, json
src/h3d/core/__init__.py	底座出口	统一暴露 core 公共符号	上述四模块
src/h3d/cli.py	命令入口	argparse 子命令 + M0 编排内核	core

2.2 模块划分原则

1. **core 不依赖业务包**：底座可被单独 import 与测试，业务子包后续只依赖 core，不反向。
2. **异常为最底层**：exceptions 被 config/metadata 引用，自身零依赖，保证任何模块抛错时错误码契约稳定。
3. **入口与逻辑解耦**：cli.py 只做参数解析与调度，业务逻辑通过 STAGE_REGISTRY 注册表挂载，M0 不直接 import 业务实现。

第 3 章 依赖关系与数据流

3.1 模块依赖关系图



依赖方向自上而下、禁止反向环，与《管线骨架构建分析》第 5.1 节「包内依赖」一致。

3.2 包间依赖矩阵

「□」表示行导入列（A → B 读作 A 导入 B）。P0 底座只涉及 core 与 cli：

导入方 \ 被导入方	core.exceptions	core.config	core.logging	core.metadata	cli
core.exceptions	—				
core.config	□	—			
core.logging			—		
core.metadata				—	
cli	□	□	□	□	—

3.3 一次运行的数据流

```
命令行:  h3d run --config configs/example.yaml
   |
   ▼
[1] load_config(yaml) ──□ core/config.py
        yaml.safe_load → 描述符逐字段校验 → Config 对象 → config_sha256
   |
   ▼
[2] setup_logging(level, log_file) ──□ core/logging.py
        控制台 + 文件(90_meta/pipeline.log) 双通道
   |
   ▼
[3] _discover_items(cfg) ──□ 显式 project.items 或扫描 input.raw_dir
   |
   ▼
[4] RunManifest.load(manifest_path) ──□ core/metadata.py
```

```
      断点续跑：已有清单则加载；配置变更则重置
   │
   ▼
[5] 编排内核：for item → for stage in STAGE_REGISTRY
      ├─ 成功：manifest.record(item, stage, result)    （先记 manifest 再落盘）
      ├─ RecoverableError：manifest.record_fail(...) → break（失败隔离）
      └─ 未知异常：同上隔离，不中断队列
   │
   ▼
[6] manifest.save(manifest_path)   （原子写：.tmp → replace）
   │
   ▼
[7] 输出汇总（总数/完成/失败/待处理）+ run_manifest.json 落盘
```

关键设计点：**先记 manifest 再落盘**（《设计书》5.4、27.1 节），保证中断可恢复、可审计。

第 4 章 core/exceptions.py — 异常层级

4.1 功能

定义管线自定义异常类层级，按《设计书》13.3 节错误码规范组织。异常实例可携带错误码（code）、阶段标识（stage）与消息，并可序列化为 dict 供 run_manifest 记录失败原因。

4.2 类层级

```
H3DError(Exception)                  E000  异常基类
├── RecoverableError(H3DError)             可恢复标记（失败隔离的判定依据）
│   ├── DataError                    E100  输入缺失/损坏
│   ├── RegistrationError              E200  配准失败
│   ├── QualityError                 E500  质检不达标
│   └── ExternalProcessError             E900  外部子进程异常
└── ConfigError(H3DError)                E001  配置缺失/非法（不可恢复）
```

4.3 关键类/函数职责

符号	类型	职责
H3DError	类	异常基类；__init__(message, code, stage)，str(exc) 输出 [code][stage] message
H3DError.to_dict	方法	序列化为 {code, message, stage?}，供 manifest 记录
RecoverableError	类	标记可恢复错误；编排器捕获后 break 实现失败隔离
ConfigError	类	E001；启动即退出（不可恢复），CLI 统一退出码 2

4.4 错误码对照表

错误码	异常类	含义	处理策略
E001	ConfigError	配置缺失/非法	启动即报错退出
E100	DataError	输入缺失/损坏	标记跳过，记入 manifest
E200	RegistrationError	配准失败	回退粗配准或告警
E500	QualityError	质检不达标	阻断交付，提示重跑参数
E900	ExternalProcessError	外部子进程异常	隔离，不影响队列其余项

第 5 章 core/config.py — 配置加载与校验

5.1 功能

采用「描述符（descriptor）元编程」实现字段级类型与范围校验，配合 YAML 安全解析，把运行参数外部化并纳入可复现体系（config_sha256）。

5.2 内部结构

5.2.1 描述符层（Field 系列）

类	职责
Field	描述符基类：__set_name__/__get__/__set__；赋值时走 _coerce（类型强制）→ _check（范围校验）
FloatField	浮点字段：强制 float，拒绝 bool
IntField	整数字段：强制 int，拒绝 bool
BoolField	布尔字段：接受 bool/0/1/"true"/"false"
StrField	字符串字段
StrListField	字符串列表：单字符串自动包装为单元素列表

5.2.2 配置段（Section 系列）

类	对应设计书模块	字段（默认值）
ProjectSection	项目元信息	name、operator、root、items
InputSection	M1 数据接入	raw_dir=data/raw、formats=[ply,pcd,obj]
PreprocessSection	M1 预处理	voxel_down_sample=0.005、normal_estimate_k=30、remove_outlier_nb=20、remove_outlier_std=2.0
RegisterSection	M2 配准	fpfh_radius=0.02、ransac_distance=0.05、icp_distance=0.02、max_iter=2000
ChangeSection	X1 多时相	enabled=false、core_radius=0.05
ColourSection	X2 色差	enabled=false
RunnerSection	横切调度	fail_isolation=true、resume=true、manifest=output/run_manifest.json

5.2.3 顶层类

符号	职责
Config	聚合七段配置；from_file/from_dict/to_dict/config_sha256
Config.config_sha256	键排序 + 紧凑 JSON 的 SHA256，作为 manifest 可复现校验依据
load_config(path)	YAML → Config（CLI 入口）
load_config_dict(dict)	dict → Config（库调用/测试入口）

5.3 校验行为

- 未知顶层字段/段内字段 → 抛 ConfigError（拼写错误即时暴露）；
- 类型不符 → 强制转换；转换失败 → ConfigError；
- 越界（min/max）→ ConfigError；
- 统一 yaml.safe_load，禁止 yaml.load（防任意代码执行）。

第 6 章 core/logging.py — 分级日志

6.1 功能

提供统一分级日志（DEBUG/INFO/WARNING/ERROR），控制台 + 文件双通道，UTF-8 编码，供 cli 与业务模块共用一套格式。

6.2 关键函数

符号	职责
setup_logging(level, log_file, name)	初始化根日志器；幂等（重复调用不叠加 handler）；自动创建日志父目录
get_logger(name)	获取命名子日志器（继承根配置）
set_level(level)	运行时调整级别（--verbose → DEBUG）

6.3 设计要点

- 控制台 handler 用私有标记 _h3d_console 识别去重；文件 handler 按目标路径去重；
- 日志文件默认落 90_meta/pipeline.log，支撑「日志集中至 90_meta」的运维要求（《设计书》28.2）
- 。

第 7 章 core/metadata.py — SHA256 与 run_manifest 溯源

7.1 功能

实现 SHA256 校验与 run_manifest.json 溯源清单，记录每次运行的输入哈希、参数快照、各阶段状态、耗时与产物路径，是「可复现 / 可审计 / 可软著溯源」的真相来源。

7.2 关键类/函数

符号	类型	职责
sha256_text(text)	函数	字符串 SHA256
sha256_file(path)	函数	分块（1 MiB）计算文件 SHA256，避免大文件整读入内存
collect_software_versions()	函数	收集 open3d/numpy/py4dgeo/colour/yaml/tqdm 版本快照，缺库记 None
StageRecord	dataclass	单阶段记录：status/inputs/outputs/elapsed_s/metrics/reason
RunManifest	类	运行清单：item/阶段读写、汇总、原子持久化

7.3 RunManifest 关键方法

方法	职责
item(item_id)	取得（必要时创建）尊像记录
record(item_id, stage, result)	记录阶段成功；metrics（如 rmse_mm）平铺进记录
record_fail(item_id, stage, reason)	记录阶段失败，item 置 fail
record_skip(item_id, stage, reason)	记录阶段跳过（可选模块关闭）
stage_done(item_id, stage)	断点续跑判定
summary()	返回 {total, done, fail, pending}
save(path)	原子写：先写 .tmp 再 replace
load(path)	加载已有清单；不存在返回空清单

7.4 run_manifest.json 结构


```
{
  "run_id": "20260824-200245-b383",
  "created_at": "2026-08-24T20:02:45+08:00",
  "config_sha256": "e25f35ab...",
  "software": {"numpy": "2.5.1", "yaml": "6.0.3", "open3d": null, "...": "..."},
  "items": [
    {
      "item_id": "rock001",
      "status": "done",
      "stages": {
        "M1": {"status": "done", "outputs": ["10_clean/rock001.ply"], "elapsed_s": 0.5, "rmse_mm": 1.2},
        "M2": {"status": "fail", "reason": "重叠不足"}
      }
```

```
    }
  ]
}
```

字段字典对齐《设计书》41.1 节：run_id / created_at / config_sha256 / stages（阶段状态） / software（依赖版本快照）；阶段指标（rmse_mm 等）与 status 平铺，与 12.1 节示例一致。

第 8 章 cli.py — 命令入口与编排内核

8.1 功能

提供 h3d 命令行入口与 M0 编排内核：解析配置、发现尊像、遍历队列、按序调用业务阶段、记录状态、支持断点续跑与失败隔离。CLI 与（后续的）GUI 共用同一内核。

8.2 子命令

命令	参数	功能
h3d init	--project <dir>	初始化项目数据目录（00_raw..90_meta）
h3d validate	--config <yaml>	校验配置并打印 config_sha256
h3d run	--config <yaml> [--item] [--dry-run] [--verbose]	运行管线（编排内核）
h3d report	--manifest <json>	打印 run_manifest 运行汇总
h3d --version	—	打印版本

8.3 关键符号

符号	类型	职责
STAGE_REGISTRY	dict	阶段注册表；业务子包经 register_stage 挂载 run_* 函数
register_stage(name)	装饰器	把业务函数挂载到 M0 编排内核
PROJECT_DIRS	list	目录契约（00_raw…90_meta）
_discover_items(cfg)	函数	显式 project.items 优先，否则扫描 input.raw_dir
cmd_run(args)	函数	编排内核：断点续跑 + 失败隔离 + 先记 manifest 再落盘
main(argv)	函数	入口：解析 → 分发 → 统一异常退出码

8.4 编排内核逻辑（cmd_run）

```
load_config → setup_logging → _discover_items
  → RunManifest.load（断点续跑；配置变更则重置）
  → for item in items:
      if resume and item.done: 跳过
      for stage, fn in STAGE_REGISTRY.items():
          if resume and stage_done(item, stage): 跳过
          try:   res = fn(item, cfg); manifest.record(item, stage, res)
          except RecoverableError:  manifest.record_fail(...); break   # 失败隔离
          except Exception:         同上（隔离，不崩溃）
      全部成功 → mark_item_done(item)
  → manifest.save（原子写）→ 输出汇总
```

8.5 异常退出码

退出码	含义
0	成功

退出码	含义
1	H3DError（运行期业务错误）
2	ConfigError（配置错误）
130	用户中断（Ctrl+C）

第 9 章 关键接口契约汇总

P0 底座对外暴露的核心接口（供后续业务子包与调用方使用）：

```
# 配置
from h3d.core import load_config, Config
cfg = load_config("configs/example.yaml")        # -> Config
cfg.preprocess.voxel_down_sample                 # 描述符校验后的字段
cfg.config_sha256                                # 规范化哈希

# 日志
from h3d.core import setup_logging, get_logger
logger = setup_logging("INFO", log_file="90_meta/pipeline.log")

# 溯源
from h3d.core import RunManifest, sha256_file
m = RunManifest(config_sha256=cfg.config_sha256)
m.record("rock001", "M1", {"status": "done", "metrics": {"rmse_mm": 1.8}})
m.save("output/run_manifest.json")

# 异常
from h3d.core import ConfigError, DataError, RegistrationError
raise RegistrationError("重叠不足", stage="M2")    # [E200][M2] 重叠不足

# 业务子包接入编排（M1~M6 后续实现时）
from h3d.cli import register_stage
@register_stage("M2")
def run_m2(item: str, cfg) -> dict:
    ...  # 输入 10_clean/<item>，输出 20_reg/<item>
    return {"status": "done", "outputs": [...], "elapsed_s": 1.2,
            "metrics": {"rmse_mm": 1.8}}
```

业务阶段统一返回结构（《设计书》附录 B）：{status: done|fail|skip, outputs: [...], elapsed_s: float, metrics: {...}}。

第 10 章 目录与文件清单

10.1 项目目录（完整软著版骨架）

```
h3d_pipeline/
├─ src/h3d/
│   ├── __init__.py          # 包入口 + __version__
│   ├── cli.py               # 命令入口（P0 底座，已实现）
│   ├── core/                # 核心底座（P0 底座，已实现）
│   │   ├── __init__.py      # 统一出口
│   │   ├── config.py        # 描述符校验 + YAML 加载
│   │   ├── logging.py       # 分级日志
│   │   ├── metadata.py      # SHA256 + run_manifest
│   │   └── exceptions.py    # 异常层级
│   ├── io/  preprocess/  register/  change/  colour/  runner/  report/
│   │   （占位：后续阶段实现）
├─ tests/test_core/          # P0 底座单元测试（已实现，31 用例）
├─ configs/example.yaml      # 示例配置
├─ docs/                     # 文档
├─ examples/  data/  output/  notebooks/
├─ pyproject.toml  environment.yml  README.md  .gitignore
```

10.2 代码规模

类别	文件数	约行数
P0 底座业务源码	7	1060
单元测试	5	351

第 11 章 测试与运行

11.1 运行单元测试

```
cd h3d_pipeline
python -m venv .venv && .venv\Scripts\activate
pip install -e ".[dev]"          # 安装 h3d 包 + pytest
pytest -q                        # 收集并运行 tests/（31 用例通过）
```

11.2 运行 CLI

```
h3d init --project ./demo_project          # 初始化项目目录
h3d validate -c configs/example.yaml       # 校验配置
h3d run -c configs/example.yaml            # 运行管线
h3d run -c configs/example.yaml --dry-run  # 只预览待处理尊像
h3d report -m demo_project/90_meta/run_manifest.json   # 查看汇总
```

11.3 测试覆盖

测试文件	覆盖内容
test_exceptions.py	错误码契约、可恢复标记、to_dict、消息格式
test_config.py	默认值、类型强制、越界/未知字段报错、config_sha256 稳定性与敏感性、to_dict 往返
test_logging.py	级别解析、幂等初始化、非法级别报错
test_metadata.py	SHA256 确定性、record/stage_done、summary 计数、manifest 往返、保留字段优先级
test_cli_orchestration.py	编排内核端到端：失败隔离、断点续跑、ConfigError 退出码

附录 A 配置字段速查

段.字段	类型	默认	范围/说明
project.name	str	""	项目名
project.root	str	"."	项目数据根目录
project.items	list	[]	显式尊像队列（空则自动扫描）
input.raw_dir	str	data/raw	原始数据目录
input.formats	list	[ply,pcd,obj]	输入格式
preprocess.voxel_down_sample	float	0.005	0.0005–0.01 m
preprocess.normal_estimate_k	int	30	3–1000
preprocess.remove_outlier_nb	int	20	1–1000
preprocess.remove_outlier_std	float	2.0	0–10
register.fpfh_radius	float	0.02	0.001–0.5
register.ransac_distance	float	0.05	0.001–1.0
register.icp_distance	float	0.02	0.0001–0.5
register.max_iter	int	2000	1–1000000
change.enabled / colour.enabled	bool	false	可选模块开关
runner.fail_isolation / runner.resume	bool	true	失败隔离/断点续跑
runner.manifest	str	output/run_manifest.json	溯源清单路径

附录 B 合规与未核实事项

- 本底座仅依赖 MIT/BSD 栈（PyYAML、NumPy 等）与标准库，不引入 GPL 库；点云读写由 Open3D（MIT）在 io 子包实现（后续阶段）。
- 软著登记版须为完全人工重写；本底座为开发基线，登记前应逐行人工重写并保留 commit 记录。
- 精度阈值（中误差≤5mm 等）以《精度与质量验收细则》最终确认值为准，本底座仅固化配置字段占位。

> 本说明书由 AI 辅助生成，供开发实施与软著登记内部参考；涉及开源协议与软著合规条款以最新官方规定为准。